Long Division and the End of Certainty: Technical Companion

Timothy Trujillo

Contents

About this companion **v**

1 Words, records, and instruction sheets **1**

2 The proof cards and their records **13**

3 Further instruction and proof examples **29**

4 Further rooms **37**

Sources and Further Reading **55**

About this companion

A direction to copy a program can fit comfortably into a story. The instructions that make copying precise take more room: the worker must recognize the characters, preserve the spaces, and distinguish a copied command from a command being obeyed. The main book follows the consequences of these abilities. This companion leaves the specifications open on the desk.

The first chapter gives the word and execution-record conventions behind `Print`, `Combine`, and `Run`. The complete addition, subtraction, and long-division sheets remain in Appendix A of the main book, together with their lookup tables and helpers. The second chapter gives the formal proof cards and the record-coding requirements used by the later arguments. It distinguishes the construction of any particular finite record from the stronger claims about records inside a model.

The third chapter follows several additional instruction and proof examples. The fourth returns to the decimal game, then develops the forcing names and Boolean truth values behind the coda. These chapters can be consulted where a question arises; they are not another stage that must be completed after the book's ending.

The mathematical status of the material matters. The P/Q argument uses the hypotheses stated in the main book: a suitable consistent proof method, effective verification of actual finite work, and a fixed interpretation supporting the combined records and their arithmetic. The details here identify those dependencies and cite the relevant representation results. They do not constitute a newly completed formalization or an independent verification of every coding obligation. Likewise, the forcing discussion retains its additional assumption of a countable transitive ground model. Moving these details into a companion changes their location, not the strength of the claims.

References to numbered chapters of the main book are identified as such. The sources at the end provide routes into the original work.

Chapter 1

Words, records, and instruction sheets

1.1 Words, returned answers, and instruction sheets

Place cards are indifferent to whether a row spells a word or gives an amount. They can follow either. Keep one card at its beginning and another just past its last character. Start a working card at the beginning. Before reading, compare its place with the saved end. If they agree, the word is finished. Otherwise read its character and move right. The same method works for an empty word because its beginning and end cards already agree.

To copy `MATH`, begin a second working card on a fresh row. Copy M and move both working cards right; then do the same with A, T, and H. The first card now agrees with the source’s end. Save the second card as the new word’s end. Appending an exclamation mark writes `!` there and moves that end card right once. The result is `MATH!`. Removing the final character moves the end card left once and erases that square, provided the word is nonempty.

To clear a word, move from its beginning to its saved end, erasing one character at a time, then copy its beginning place onto its end card. To compare two words, advance a working card through each. At each step, test whether either card has reached its end. Two ends together mean equal words; one end alone means unequal words. Before the ends, compare the two characters by the finitely many character branches. A mismatch ends the comparison; a match moves both cards right. The procedure finishes because the words are finite.

To count the characters, begin a separate number row with 0 and add one each time a character is passed. Joining two words copies the first to a fresh row, then continues copying the second at the place where the first copy ended. These procedures also

supply copying, counting, joining, and comparing for a word of digits. A numeral is normalized by removing its unnecessary initial zeros while retaining one zero when they are all zero, just as on an arithmetic answer row.

`Print(s)` is the name of a finite program-building task. It returns an instruction sheet that, when followed, returns the fixed word s. `Combine(p,q)` builds a sheet that first follows p, gives its returned word to q, and then follows q. `Run(p,x)` actually follows the sheet p with supplied word x. Building a sheet is a finite copying task even when following it would continue forever. The person copying “do this again” can finish copying.

For example, `Print(MATH)` builds a sheet whose returned word is `MATH`. A second sheet can copy its supplied word, append `!`, and return the new word. Their combination returns `MATH!`. The returned word occupies an ordinary work row. Digits settled in a displayed answer are separately identified by their place, so a returned program text is available for reading without being mistaken for decimal output.

1.2 The exact form of words and execution records

Copying instructions brings punctuation into the calculation. A space inside a word, a blank square, and the gap after a word cannot simply be treated as the same nothing. The conventions below give each a definite job. Arithmetic rows still use one ordinary digit per square; general words use a fixed encoding that distinguishes every permitted character, including spaces and punctuation.

Character squares and their saved ends

A code table lets us copy letters with the same two marks used for any other coded text. In this table each character has seven squares, each containing zero or one. Such a zero-or-one mark is called a *bit*. The table is the familiar convention called ASCII; using it here requires lookup, not arithmetic in a new number system.

For example, the entry for `M` is `1001101`. Write those seven marks in seven consecutive squares. The next letter starts in the eighth square. The four letters of `MATH` become

M	A	T	H
1001101	1000001	1010100	1001000

The dividing lines show the groups; they are not extra marks on the row. To read the word back, take seven squares at a time and find the matching entry. The first group gives `M` again. A space has its own seven marks, `0100000`; it is different from unused, blank paper. We use the line-feed entry for a new line.

The complete table below supplies every character permitted in these general words: the printable characters, space, and new line. Other seven-square patterns are rejected.[1]

[1]The character assignments are tabulated in Vint Cerf, *ASCII format for Network Interchange*, RFC 20 (1969), rfc-editor.org/rfc/rfc20.html.

mark	code	mark	code	mark	code
new line	0001010	?	0111111	_	1011111
space	0100000	@	1000000	‘	1100000
!	0100001	A	1000001	a	1100001
"	0100010	B	1000010	b	1100010
#	0100011	C	1000011	c	1100011
$	0100100	D	1000100	d	1100100
%	0100101	E	1000101	e	1100101
&	0100110	F	1000110	f	1100110
’	0100111	G	1000111	g	1100111
(	0101000	H	1001000	h	1101000
)	0101001	I	1001001	i	1101001
*	0101010	J	1001010	j	1101010
+	0101011	K	1001011	k	1101011
,	0101100	L	1001100	l	1101100
-	0101101	M	1001101	m	1101101
.	0101110	N	1001110	n	1101110
/	0101111	O	1001111	o	1101111
0	0110000	P	1010000	p	1110000
1	0110001	Q	1010001	q	1110001
2	0110010	R	1010010	r	1110010
3	0110011	S	1010011	s	1110011
4	0110100	T	1010100	t	1110100
5	0110101	U	1010101	u	1110101
6	0110110	V	1010110	v	1110110
7	0110111	W	1010111	w	1110111
8	0111000	X	1011000	x	1111000
9	0111001	Y	1011001	y	1111001
:	0111010	Z	1011010	z	1111010
;	0111011	[	1011011	{	1111011
<	0111100	\	1011100	|	1111100
=	0111101	]	1011101	}	1111101
>	0111110	^	1011110	~	1111110

A general word with n characters occupies $7n$ squares: seven times as many squares as letters, punctuation marks, and spaces. For MATH, that is 28 squares. If its first square has column zero, its last has column 27, and its saved end card records column 28, just past the writing. A character move advances seven squares. Character copy writes those seven marks before advancing to the next character; comparison checks all seven. Removing a character erases its seven-square block.

Counting still increases once per decoded character. Consequently `MATH!` has length five. A decimal digit in a general word has a seven-square code, whereas a digit on an arithmetic row has its familiar one-square form. Ten finite branches convert one form to the other. The kind of row is specified when a routine is prepared.

For general program runs, use separate pages for the supplied word, the returned word, the displayed answer, and the remaining work. The supplied and returned words have fixed beginning places and saved end cards. The returned word initially is empty. No end mark is written into the character row. Appending uses the saved end place; clearing sets that place back to the beginning.

Settled output

For the standardized printers used in the later arguments, number the successive digit places on the answer row $0, 1, 2, \ldots$. An output card begins at the first of these places. `Emit(d)` writes digit d there and moves the card one square right. Its ten possible branches are fixed basic writes and moves. `EmitString(s)` follows the supplied finite word and performs the corresponding branch for each digit.

A decimal display may place its initial `0.` in the two squares immediately to the left. The successive digit places then hold its fractional digits. For a numeral printer, those same successive places hold the numeral itself. The later construction specifies which of these readings is intended. In either case, an emitted digit is never revised. The schoolroom division sheet has its ordinary display, including its integer part and point; its fractional digits are the digits after that point.

Lists of words

A record containing several words first gives the number of parts followed by a colon. Each part then gives its character length, a colon, and its characters. For example, the words `8` and `123` are recorded as `2:1:83:123`. After reading `1:`, the worker takes exactly one character; after `3:`, exactly three. A colon among those specified characters belongs to the data. Counts have ordinary

decimal notation without leading zeros. The worker can count down with the subtraction sheet to locate the end of each part. Missing characters, invalid lengths, or extra characters after the complete record cause rejection. The whole record is stored using the character encoding above. Lists nested inside lists use the same rule.

Read the example once with a finger, separating its jobs:

part read	what the worker now knows
`2:`	there are two words to collect
`1:`	the first word has one character
`8`	the first word is eight
`3:`	the second word has three characters
`123`	the second word is one, two, three

The character `3` after `8` starts the next length; it does not continue the first word, because that word's one character has already been taken. If a word itself is `A:B`, its part is `3:A:B`: after reading the length, take three characters, including the colon between the letters. The length tells us when to treat a colon as data rather than as a separator. Even punctuation sometimes has to wait its turn to give an instruction.

Recording one moment and following one line

A configuration records the current instruction number, the positions on the program's finitely many named cards, and the finite list of nonblank paper squares. A square entry gives (p, x, y, a): page, column, row, and mark. Put entries in increasing page order, then column order, then row order, with no duplicate positions. The instruction sheet fixes the order of the named cards in their list. All coordinates are written as signed decimal numerals.

To follow `Write`, find the position named by the relevant card, then insert, replace, or erase its square entry. To follow `Move`, add the three stated fixed shifts to that card's coordinates. To follow `Place`, copy one card's three coordinates to the other. A square test searches the finite list, reading blank when the position has no entry. A same-place test compares the two triples. A jump

changes the instruction number. A halt is recognized directly from the indicated line.

For signed addition, two numbers with the same sign add their magnitudes and keep that sign. For different signs, subtract the smaller magnitude from the larger and keep the larger magnitude's sign. Equal magnitudes give zero without a minus sign. Thus adding 3 to −2 gives $3 - 2 = 1$. Comparing signed numbers puts negative numbers below nonnegative numbers; two negative numbers reverse the order of their magnitudes. These finite sign choices and the arithmetic sheets supply the coordinate updates and the ordered insertion of new square entries.

Every one of these searches and updates finishes on a finite record. They provide `InitialConfiguration`, which prepares the first record; `Step`, which follows one instruction; `NextConfiguration`, which takes that step's new record; and `Halted`, which tests for a halt. `Step` can also report a newly settled output digit. A finite run is a finite list of these configurations, each itself made of finite lists.

To follow a supplied program for many steps, keep its record on one's own work pages, make one update, and repeat. This gives an interpreter. Its own instructions and work occupy different places from the encoded squares of the program it is following. If the supplied program orders an erasure, the interpreter changes the corresponding entry. It has no reason to rub out its own instructions. The separation is as practical as keeping a worked example beside the page on which we are checking it.

Malformed answers and arbitrary programs

Nothing stops us from writing an instruction sheet that makes a mess. A program can leave half a character, put its end card on the wrong row, or alter a digit it was supposed to have settled. Its raw steps still have definite meanings. We need to recognize what it has actually done without quietly tidying the answer on its behalf. At a halt, inspect the finite record of the returned-word page and its saved end card. A valid end is on the specified row, at a nonnegative multiple of seven from the fixed beginning; the intervening squares contain exactly complete permitted character codes, and no other nonblank square occurs on that page. The finite

end coordinate gives a finite search bound, and the finite square list supplies the check for other marks. An invalid arrangement returns a distinguished “malformed result” report. The actual halt is recognized in either case.

Likewise, a settled-output write must put the next digit in its designated place without changing any earlier output square. The interpreter keeps its own record of the next permitted place. If a simulated program violates the rule, the interpreter records the violation and suppresses subsequent output forwarding. It continues to follow every raw instruction of that program. A private simulation testing whether a program halts can ignore output reports entirely. Thus these checks never turn a nonhalting raw program into a halting one, or prevent the recognition of an actual halt.

Constructing exactly the same instruction sheet

Two sheets can do the same thing without being the same sheet. The self-description argument needs the stronger agreement: every character must match. Here are the complete compact wrappers for `Print` and `Combine`. They are macro sheets: `Load` and `Run` name the finite copying and simulation procedures described above, not additional elementary operations. Each displayed line has one new-line character after it. Apart from characters inside a literal field, there are no spaces on these wrapper lines. Case, punctuation, line order, and the names of boxes are exactly as printed.

A literal field has the form `n:s`. The numeral n is the number of characters in s, written without redundant zeros; the empty field is `0:`. After the colon, take exactly n characters. All of them are data, including parentheses, commas, colons, spaces, or new lines. Only after those characters does the closing parenthesis belong to the enclosing command. Thus `Load(RESULT,3:A:B)` copies the three-character word `A:B`. The word `A)` has length two and gives the line `Load(RESULT,2:A))`: the first closing parenthesis is copied data, the second closes the command. Nested literal fields therefore do not require a new kind of quotation.

To prepare `Print(s)`, replace `n:s` in this fixed wrapper by the length and complete text of s:

```
Load(RESULT,n:s)
Halt
```

For the one-character word `A`, the complete compact sheet is

```
Load(RESULT,1:A)
Halt
```

`Load` replaces the destination word, resetting its beginning and end cards, then copying exactly the literal characters. It advances the end by seven physical squares per character and returns to the following wrapper line. Consequently this sheet returns `A` on every input. It emits no settled digits. Its elementary expansion clears the result row, writes the seven bits `1000001`, saves the end seven squares after the beginning, and halts.

To prepare `Combine(p,q)`, replace `m:p` by the length and complete text of p, and `n:q` by the length and complete text of q, in precisely this wrapper:

```
Load(P,m:p)
Load(Q,n:q)
Run(P,INPUT,Y,LEFT)
Run(Q,Y,RESULT,RIGHT)
Halt
```

For the elementary texts `0: Halt` and `0: Goto 0`, neither with a final new line, this produces

```
Load(P,7:0: Halt)
Load(Q,9:0: Goto 0)
Run(P,INPUT,Y,LEFT)
Run(Q,Y,RESULT,RIGHT)
Halt
```

The preparation copies five wrapper lines and two finite literal texts. It runs neither supplied program. In execution the first simulation returns an empty word; the second repeats its jump forever. Preparing these five lines a second time gives the same

literal fields `7:0: Halt` and `9:0: Goto 0`, followed by the same three lines and the same new lines. Every character agrees.

Arguments, work areas, and returns. Every invocation has its own frame: a separate record containing the words `INPUT`, `RESULT`, `P`, `Q`, and `Y`, together with the simulated records `LEFT` and `RIGHT`. `INPUT` is a copy of the caller's supplied word. All other word boxes start empty; the two simulation records start unused. These are named entries in a finite record, stored with the list conventions above. A nested invocation gets a new record, rather than reusing its caller's boxes. Its frame is kept with the caller's saved next-line number until it returns. This is the paper counterpart of keeping one unfinished worksheet while opening another.

`Run(C,X,Z,W)` reads the program text in box `C` and a copy of the input word in box `X`. It initializes the private record `W` with that program's starting cards, supplied word, empty returned-word row, and otherwise blank simulated paper. It repeatedly applies the one-step rule. A valid halt copies the entire returned word, including its end position, to `Z`, then resumes at the wrapper line immediately following this `Run`. It does not jump to the caller's own halt. A run that continues forever never returns. `P` and `Q` remain the same supplied texts; simulated erasures alter only the corresponding record in `LEFT` or `RIGHT`.

Settled digits from a valid simulated output are forwarded in order to the invocation's output channel. The second run continues after all digits forwarded by the first; its own recorded output positions still start at zero. Temporary word boxes are never forwarded as digits. A malformed returned word stops the wrapper with the same distinguished malformed-result report defined above; it does not start the next simulation with an invented input. A simulated output violation is handled by the earlier suppression convention while raw simulation continues. Reaching the wrapper's own `Halt` returns its `RESULT` to its caller, or ends the whole calculation when there is no caller.

Compact sheets and elementary descriptions. The placeholders `m:p`, `n:q`, and `n:s` are preparation slots, not characters left in the

finished sheet. Counting and copying supply every character in each finished compact wrapper. Inside a literal, the supplied text is preserved verbatim; it is never renumbered, reformatted, or expanded by the copying step. When the supplied text includes a new line, that character contributes one to its length even though its seven-square code occupies seven physical squares.

As elsewhere in the book, the complete program description means the elementary instruction sheet after expansion. Fix the elementary copying and interpreter routines once, before any texts are supplied. Expand wrapper lines in their printed order, and number new instruction labels in increasing order as they are emitted; use the same fixed naming of physical marks and cards on every occasion. A general `Call F(arguments,result)` makes a private frame for the fixed routine `F`, copies the listed arguments into its input fields in that order, saves the following caller line, and resumes there after copying the returned result. It does not substitute argument characters into instructions to be obeyed. More than one incoming word is packed as a list of words using the specified length fields.

The complete elementary expansion of the interpreter is not printed. The wrapper and all its literal slots are printed: the reader can assemble and compare those compact sheets, and follow their effects with the simulation rules. Equality of compact sheets implies equality of their elementary expansions under the one fixed expansion procedure. In particular, if the supplied text is the completed elementary white sheet of Chapter 4 (main book), both preparations copy that same text into the same slots. Both give the identical expanded blue sheet. This is the textual identity needed in the self-description argument; it does not depend on different programs merely having the same behavior.

Chapter 2

The proof cards and their records

A proof leaves marks behind, including marks made under assumptions we later reject. These cards specify which marks a checker may keep using. The short primitive cards come first, followed by the familiar arguments assembled from them. The example of a worker writing 7, moving right, and halting then lets us see what the resulting language says about an actual computation.

There is a larger mathematical result behind the treatment of records of arbitrary length. "Why sets can hold an ordered record" separates what we build directly by adjunction from the uniform coding theorems we use. Those theorems are stated and cited here, rather than proved in full. Finally, the common-record clauses keep the two integer outputs and their long division together, as Chapter 7 (main book) requires.

2.1 The small alphabet

Besides membership and equality, the primitive logical signs are $\wedge$, $\neg$, $\exists$ and the contradiction marker $\bot$. Variable names can be written as `v` followed by an ordinary numeral, as in `v0`, `v1`, `v17`. The finitely many digit marks therefore provide as many distinct names as a proof needs. A *term* is an expression naming an object. In our basic language it is just a variable, or a constant once constants have been added. A variable leaves its object available for assignment; an added constant is a name whose assigned object is kept fixed within a model.

Our two set axioms are relational. They assert that suitable objects exist; they do not identify a unique empty object or a unique result of adjunction. The notation E for an empty object and $J(s,t)$ for an adjoined object means that witnesses have been chosen. For any particular finite construction, this notation can be expanded

into successive existence statements and fresh witness branches. For example, $J(E, a)$ tells us to use adjunction to choose an object whose members are those of the chosen empty object together with a. Its members are consequently exactly a. The notation records that choice, not a uniqueness claim. We use it only as this abbreviation for the successive witness choices.

The starting formulas are $s \in t$, $s = t$, and $\bot$. The first two are called *atomic*: they have no smaller statements joined inside them. The contradiction marker records failure. Parentheses and the logical signs build larger formulas. A formula is any grammatically permitted statement pattern. A *sentence* has no free variables left awaiting assignment. For example, $x \in b$ contains free variables, whereas $\forall b \exists x (x \in b)$ is a sentence. Being a sentence says nothing about being true. We use the abbreviations

$$\begin{aligned} A \to B &:= \neg(A \wedge \neg B), \\ A \vee B &:= \neg(\neg A \wedge \neg B), \\ \forall x\, A &:= \neg \exists x\, \neg A, \\ A \leftrightarrow B &:= (A \to B) \wedge (B \to A). \end{aligned}$$

The two AS axioms are the closed sentences Empty and Adjunction from Chapter 5 (main book), expanded by these conventions. The abbreviation expansion is mechanical.

Each node of a proof tree records a formula, the rule used there, its premise nodes, and any substitution or discharge labels. Each occurrence of an assumption gets its own label. This is how the checker follows what a line depends on: it brings the premises' open assumptions forward, removing only the labeled occurrences the rule explicitly discharges. A line in an unrelated closed branch remains unavailable, however convenient its conclusion would be. A theorem has no open assumptions except the designated axioms.

For example, a conjunction node may record premises $e \in c$ and $b \in c$, conclusion $(e \in c) \wedge (b \in c)$, and rule "keep both." The checker matches each conjunct to its corresponding premise and carries both premises' open-assumption lists into this node. Replacing only the second conjunct by $c \in b$ makes the node fail. No discharge occurs at this card: even a correctly matched conjunction still depends on the assumptions used to obtain its parts.

2.2 The primitive cards

Keep both; take either.

$$\frac{A \qquad B}{A \wedge B}, \qquad \frac{A \wedge B}{A}, \qquad \frac{A \wedge B}{B}.$$

The open assumptions of a combined derivation are the union of those needed by its premises.

Find a contradiction; reject a supposition.

$$\frac{A \qquad \neg A}{\bot}, \qquad \frac{\begin{array}{c}[A]^i \\ \vdots \\ \bot\end{array}}{\neg A}.$$

The second card discharges precisely the assumption occurrences labeled i. Other assumptions remain open.

Classical reductio.

$$\frac{\begin{array}{c}[\neg A]^i \\ \vdots \\ \bot\end{array}}{A}.$$

Discharge the designated occurrences. From an already established contradiction one may derive any A by opening an unused assumption $\neg A$, repeating the contradiction, and applying reductio. Double-negation elimination is also derived: assume $\neg A$, contradict $\neg\neg A$, and conclude A.

Give an example.

$$\frac{A[t/x]}{\exists x\, A}.$$

The notation $A[t/x]$ means replacing the free occurrences of x by the term t. The substitution must not cause any variable of t to become bound. For example, if $A(x)$ is $x \in b$ and t is a, then $A[a/x]$ is $a \in b$. The slash means "put the expression on the left in the free places named on the right." It is not division. If $A(b)$ is $\exists x(x \in b)$, replacing free b by x first requires renaming the bound letter. Write $\exists z(z \in b)$, then obtain $\exists z(z \in x)$. The new free name still names

the bag, rather than becoming the unknown member sought by the existential sign.

Use an unknown witness.

$$\dfrac{\exists x\, A \qquad \begin{array}{c}[A[a/x]]^i \\ \vdots \\ C\end{array}}{C}.$$

The temporary variable a is absent from $\exists x\, A$, from C, and from every undischarged assumption other than the designated copies of $A[a/x]$. Discharge those copies. Rename bound variables beforehand if necessary to make the temporary name entirely fresh. A fresh-constant version gives the same rule if the language extension is recorded.

Equality.

$$\dfrac{}{t = t}, \qquad \dfrac{s = t \qquad A[s/x]}{A[t/x]}.$$

Here $A(x)$ is a displayed template; both substitutions must be capture-free. Its chosen free occurrences of x specify which occurrences are being replaced. Bound occurrences retain their original scope.

For every substitution, the checker parses the enclosing quantifiers. It may first rename all bound variables to fresh indexes larger than every index in the formula and inserted term. It then replaces only the designated free occurrences. These finite scans check the scope conditions.

2.3 Assembled cards

The primitive cards can be joined into the familiar moves below. For example, "if" gathers a short argument under an assumption and then closes that assumption. The expansions show where each temporary assumption goes, so a proof written with the convenient larger cards still uses exactly the same calculus.

If: introduction. Suppose a derivation of B has additional labeled assumptions A. Assume $A \wedge \neg B$; take A, feed it into the

given derivation, and take $\neg B$ from the conjunction. The resulting contradiction gives $\neg(A \wedge \neg B)$. The assumptions A used in the supplied derivation have been replaced by consequences of the newly discharged conjunction. Other assumptions survive.

If: use. From $\neg(A \wedge \neg B)$ and A, assume $\neg B$, join it to A, and derive a contradiction. Reductio gives B.

Or: give either alternative. From A, assume $\neg A \wedge \neg B$, extract $\neg A$, and contradict A. Discharge the conjunction to obtain the abbreviation $A \vee B$. The other side is symmetric.

Cases. Suppose $A \vee B$, a derivation of C from additional assumptions A, and a derivation of the same C from additional assumptions B. Assume $\neg C$. Inside that branch, assume A, use the first derivation, and derive $\bot$; discharge A to obtain $\neg A$. Repeat with B to obtain $\neg B$. Their conjunction contradicts $A \vee B$. Reductio discharges $\neg C$ and yields C. Precisely the designated case assumptions are discharged; all other assumptions in both case derivations remain open.

Every: use a particular object. Given $\neg\exists x \neg A$, assume $\neg A[t/x]$. Give t as an example for $\exists x \neg A$, derive contradiction, and conclude $A[t/x]$ by reductio. Substitution remains capture-free.

Every: prove an arbitrary instance. Suppose $A(a)$ has been derived, where a is not free in any open assumption and is free for the indicated substitution. Choose a fresh variable b and rename the arbitrary parameter throughout that derivation to obtain $A(b)$. Now assume $\exists x \neg A(x)$. Use a fresh witness branch $\neg A(b)$, derive contradiction with $A(b)$, and eliminate the witness to obtain $\bot$. Discharging the existential assumption proves $\neg\exists x \neg A(x)$. Bound variables may be renamed beforehand so that b is absent everywhere required by the witness card. This is the usual universal-introduction rule with its exact open-assumption restriction.

Soundness follows by checking that each primitive card preserves truth under its open assumptions; reductio uses classical truth values. The assembled rules therefore preserve truth as well. The Henkin construction uses the derived implication and universal rules just established.

2.4 Checking and enumerating

To put a proof tree on a row of paper, use balanced delimiters to mark its branches and record each formula, rule label, premise subtree, and discharge instruction. The checker first parses the tree, then works from leaves to root. At each node it checks the rule pattern and substitution and calculates the assumptions still open. A forbidden witness name or an unsupported axiom ends the attempt. The requested conclusion is accepted only when every check has passed. All the scans are through the finite record supplied, so the checking finishes.

The search feeds this checker every finite string of the proof alphabet, ordered first by length and then alphabetically. It keeps the successful proofs. An endless mechanically generated list of additional axioms is allowed: an axiom leaf must then bring a finite record showing its axiom being produced by that list's enumerator. The checker examines the attached record. It never has to wait to see whether a missing axiom will someday appear. Each certificate is therefore decidably checkable, and every valid certificate eventually has its turn in the search.

2.5 Why sets can hold an ordered record

Adjunction constructs finite containers. To represent an ordered list, we attach a position label to each entry. Begin with

$$\mathrm{One}(u, x) \quad \text{as} \quad \forall z\,(z \in u \leftrightarrow z = x),$$

and

$$\mathrm{Two}(u, x, y) \quad \text{as} \quad \forall z\,(z \in u \leftrightarrow (z = x \vee z = y)).$$

Empty and repeated adjunction supply such objects. Define a pair-code relation by

$$\mathrm{Pair}(p, x, y) \quad \text{as} \quad \exists u \exists v\,(\mathrm{One}(u, x) \wedge \mathrm{Two}(v, x, y) \wedge \mathrm{Two}(p, u, v)).$$

This nested-set representation of an ordered pair is expressed as a relation. Since AS lacks extensionality, different objects can code

the same pair. Equality of represented pairs means agreement of their entries. Each code determines those entries: x is the unique member common to all members of p; y is the other member appearing there, if there is another, and otherwise equals x. This description also handles distinct singleton objects with identical membership. Consequently $\mathrm{Pair}(p, x, y)$ and $\mathrm{Pair}(p, x', y')$ imply $x = x'$ and $y = y'$.

A word such as 707 needs its two sevens kept in their separate places. For any given ordinary finite string, choose distinct position labels, attach each symbol to its label with a pair code, and collect the codes by adjunction. Equality of represented strings means agreement of the symbols at corresponding positions. Repeated symbols now keep their places, and different containers with the same represented content can serve as representatives of the same record.

There is an elementary supply of as many distinct labels as any particular finite calculation needs. In the notation with chosen witnesses, put $n_0 = E$ and $n_{i+1} = J(n_i, n_i)$. For every actual finite index, the axioms verify that the members of n_i are precisely $n_0, \ldots, n_{i-1}$, and that these finitely many labels are distinct. Ordinary induction outside AS describes how to build these proofs: each ordinary finite index receives its own finite AS proof.

To quantify over records of arbitrary internal length, we use the established interpretation in AS of ordered arithmetic and sequence coding. In an interpretation, a definable collection of objects, sometimes with an equivalence relation identifying alternate codes, serves as the domain of the represented strings or numbers. Quantifiers in the represented language are restricted to that domain. The arithmetic and sequence laws, translated into statements about these objects, are theorems of AS.[1]

Here is the quantifier restriction explicitly. Suppress the details of the chosen codes and write $\mathrm{Num}(x)$ for its domain formula, $\mathrm{EqNum}(x, y)$ for equality of represented numbers, and

[1] Visser, "The Arithmetics of a Theory," Section 2.4, p. 85, explains sequentiality through direct interpretation of AS and the resulting interpretation of weak bounded arithmetic with sequence coding. The indicated interpretation of S^1_2 supplies the ordered arithmetic and sequences used here.

Succ(x, y) for its successor relation. These names abbreviate fixed membership-language formulas supplied by the interpretation, not extra primitive symbols of AS. In this schematic notation the number-language sentence "every number has a successor" becomes

$$\forall x\big(\mathrm{Num}(x) \longrightarrow \exists y(\mathrm{Num}(y) \wedge \mathrm{Succ}(x, y))\big).$$

The outer quantifier examines objects of the AS model; the implication filters out objects that are not number codes. For each code that passes, the inner quantifier requires a number code passing the successor test. The translation of a number-language equality $a = b$ uses EqNum(a, b), which need not be literal equality of the code objects. The interpreted laws ensure that changing to an equivalent code does not change the represented arithmetic. If the chosen interpretation uses tuples as codes, the displayed variables stand for those tuples and their quantifiers for the corresponding blocks of quantifiers; the filtering works in the same way.

Thus the represented number domain consists of the passing codes, identified when they represent the same number. It is not the collection of all bags. The particular labels $n_0, n_1, n_2, \ldots$ constructed above show how to distinguish positions in each ordinary finite record; that construction alone neither defines this domain nor proves that it contains only the numbers named by ordinary numerals. In a model the interpreted domain can have numbers beyond every ordinary numeral, and its translated quantifiers range over those numbers as well.

The representation results cited in Chapter 5 (main book) give mechanical recipes for formulas describing finite instruction texts and accepting computation records. They have the following property: from an actual successful finite run of a given machine and input, one can mechanically obtain an AS proof of the translated sentence asserting that such a run exists. Translating the representation of finite computations through the interpretation just described supplies this positive finite-trace property.

Keep the three contributions separate as we return to the computation. Adjunction gives each particular ordinary finite record a container. The interpretation theorem supplies uniform formulas for the coded domain, sequences, and their operations, with proofs

of the required laws. The finite-trace property gives a proof of an output assertion for each actual successful run. Building one container, or explaining how to build arbitrarily many particular containers, does not by itself establish the latter two contributions. In particular, our external induction describing the label construction has added no induction axiom to AS.

2.6 Configurations, traces, and the output predicate

A configuration code records the instruction label, the worker's named place cards, and a finite table of marked squares. Each place card records a triple (p, i, j): page, column and row. A marked-square entry gives such a triple together with the symbol written there. Entries are ordered by their coordinates, duplicate coordinates are disallowed, and every omitted square is blank. Coordinates are finite integer strings under the fixed writing convention. The input, returned result and permanent decimal output have their specified places in the record. At any actual finite stage there are only finitely many written symbols and place cards to record.

The relation Step(e, c, d) reads the instruction selected in c and checks the corresponding change in d:

- `Move(A,dp,dx,dy)` adds the three indicated displacements to the coordinates on card `A`.
- `Place(A,B)` copies the coordinates from card `B` onto card `A`.
- `Write(A,s)` puts symbol `s` in the square named by card `A`; writing blank removes that square's marked entry.
- `If(A,s) Goto L` chooses between line `L` and the following line by comparing the symbol at `A`'s square with `s`.
- `IfSame(A,B) Goto L` makes the same choice by comparing the coordinate triples on the two cards.
- `Goto L` selects line `L`; `Halt` marks a completed computation.

The first three instructions proceed to the following line. Every clause also checks that all components outside the specified change agree. Coordinate arithmetic, copying and comparison are finite string operations. Abbreviated tasks use the fixed expansion into these instructions; a record follows the expanded sheet. A trace begins with the prescribed input and place cards, continues through configurations linked by Step, and, for a completed computation, ends at `Halt`. Thus the computation is recorded as a sequence of sequences: successive configurations of the paper and the worker's places.

Our little write–move–halt program makes the checks visible. Card A starts at $(0, 0, 0)$ on blank working paper; the instructions are `Write(A,7)`, `Move(A,0,1,0)`, `Halt`. There are three configurations at instruction boundaries. First the marked entry $((0, 0, 0), 7)$ is added and the instruction label advances. Next that entry stays put, the card changes to $(0, 1, 0)$, and the label advances to `Halt`.

Try changing the mark's coordinate along with the card. The checker rejects the certificate: moving attention does not move the mark. Try adding an unexplained second mark, and the unchanged-component requirement rejects it. A correct-looking answer cannot excuse either alteration. The clauses of Step check how the paper changed.

For use across models, the interpretation must prove that the initial configuration has the prescribed decoded data and that each nonhalting instruction has a unique decoded successor, with the unchanged-component clauses above. Uniqueness is up to represented equality, not literal identity of code objects. The interpreted arithmetic and sequence laws supply the comparisons and record access used in these clauses. For each ordinary finite number of transitions, repeated application therefore fixes the decoded configurations to be the ordinary ones. A record cannot end at such a stage unless its selected instruction is `Halt`. This ordinary-stage fidelity uses those representation laws and the deterministic definition of Step. It does not follow from consistency alone or merely from the positive finite-trace promise. Nor does the external repetition assert that every internally finite record has ordinary length.

Formally, the abbreviation $\mathrm{Out}(e, w)$ is obtained by applying the fixed coding interpretation to the assertion that such an accepting record exists with final output w. The checker and the programs that print formulas use the same coding, which remains fixed across models. An actual run supplies the record; the representation theorem supplies a formal proof. A model may also satisfy the same existential formula through a record with nonstandard interpreted length, extending beyond every ordinary finite record we can put on paper.

For the final long division, fasten the relevant records together. The certificate carries *one common witness*: both completed program records, their numeral outputs, and a single division record. The division record is a finite initial segment of the perpetual decimal printer, ending at a digit boundary with remainder zero. Further zeros may be included after the first zero remainder. Every requirement on the first k digits refers to this same witness. Thus asking for another digit keeps us with the same integer outputs and the same division, rather than quietly substituting a different calculation.

Each numeral output includes an evaluation chain: begin at zero and, on reading a digit d, replace the previous value v by $10v + d$. The last values are the represented integers p, q, and the record requires $q > 0$ and $0 \le p < q$. The certificate must include these chains and their values in the interpreted domain alongside the numeral strings.

The division component also carries running arithmetic witnesses A_i, B_i, R_i. An index is a position label: the subscript i says how many answer digits have been recorded. A_i is those first i digits read as an integer, $B_i = 10^i$ is 1 followed by i zeros, and R_i is the remainder after that digit. At position zero no digits have yet been written, so $A_0 = 0$, $B_0 = 1$, and $R_0 = p$. These letters translate Chapter 2 (main book)'s notation: its accumulated answer B is our A_i, and its factor 10^k is our B_i, with $k = i$.

For a separate illustration of these record checks, take $p = 3140, q = 10000$. The first digit is 3. Thus $A_1 = 3, B_1 = 10, R_1 = 1400$, and the check is

$$3140 \times 10 = 10000 \times 3 + 1400.$$

The next digit is 1. Hence $A_2 = 31$, $B_2 = 100$, $R_2 = 4000$, and

$$3140 \times 100 = 10000 \times 31 + 4000.$$

Both remainders are nonnegative and smaller than 10000. Reading $i + 1$ in the general rule means moving to the next such line; d_{i+1} is the new digit on that line. The conditions are

$$\begin{aligned} B_0 = 1, \quad A_0 = 0, \quad R_0 = p, \\ B_{i+1} = 10B_i, \quad A_{i+1} = 10A_i + d_{i+1}, \\ pB_i = qA_i + R_i, \qquad 0 \le R_i < q. \end{aligned}$$

The final recorded remainder is zero. The definition of a valid record requires these equations at every recorded position, including positions in a nonstandard trace. Every actual record produced by the construction has these witnesses. The integer checks let us compare the ratio p/q with decimal endpoints without performing an endless division. Technically, ratios are formed from the interpreted ordered integers in their *fraction field*: a fraction is a numerator and a nonzero denominator, equal fractions are identified by cross-multiplication, and sums and products use the familiar fraction rules. The ordered arithmetic supplied by the interpretation justifies those rules. For positive denominators, for example, comparing p/q with $31/100$ means comparing $100p$ with $31q$.

Each prefix formula includes the preceding one as a conjunct. Let $\mathrm{Digit}_i(D, d)$ say that the output string in the division component of D has digit d at the position named by the ordinary index i, using the fixed sequence interpretation. For the empty word put $\mathrm{Prefix}_\varnothing(D)$ equal to $D = D$. The quantities p and q below are the numerical values decoded from this same record D. In the fully expanded formula their codes are bound together with the clauses identifying the record's two numeral outputs; they are not additional freely chosen numbers. If s has ordinary length k, define the formula for its extension by the digit d to be

$$\begin{aligned} \mathrm{Prefix}_{sd}(D) := \ & \mathrm{Prefix}_s(D) \wedge \mathrm{Digit}_{k+1}(D, d) \\ & \wedge \big(q a_{sd} \le 10^{k+1} p < q(a_{sd} + 1)\big). \end{aligned}$$

The joined letters sd mean the word s with the single digit d appended; they do not mean multiplication. For example, take $s = 3$ and $d = 1$, so $sd = 31$, $k = 1$, and $a_{sd} = 31$. The formula keeps the requirement for the first digit 3, adds the requirement that digit 2 is 1, and checks

$$31q \leq 100p < 32q.$$

With $p = 3140, q = 10000$, this says $310000 \leq 314000 < 320000$, which is true. The earlier prefix requirement has remained part of the statement, so taking that part back out recovers the first-digit assertion.

Here a_{sd} is the ordinary integer value of the finite word sd. Compute 10^{k+1} and a_{sd} while preparing the formula, then insert their fixed ordinary numerals. This external recipe produces one finite formula for each ordinary finite word. Every longer-prefix formula entails the earlier ones by repeated conjunction elimination. The inequalities place the represented ratio in the requested decimal interval. The common record has one numeral pair and one digit string; the interpreted sequence relation supplies unambiguous entries on its domain. Its length must be sufficient for the finitely many displayed entries in any one prefix requirement.

If $\mathrm{Record}_e(D)$ specifies the shared program and division record, the certificate controller can search for negations of

$$\varphi_s(e) := \exists D \left(\mathrm{Record}_e(D) \wedge \mathrm{Prefix}_s(D)\right).$$

The controller's response to a forbidden prefix produces the two numeral outputs and a division record with that prefix. The required verification property is this: from such an actual valid common record with prefix s, a mechanical translation supplies a T-proof of $\varphi_s(e)$. The fixed AS computation interpretation supplies this property for the combined record checker. Any other proof methodology used in the finale must likewise verify these common records, as well as the individual outputs. The resulting proof contradicts a certificate excluding that prefix. Having proved that $T + \varphi_s(e)$ is consistent for every standard s, one may add a common constant naming D and use the nested prefix conditions in compactness. The witness then remains common across all ordinary finite beginnings.

2.7 The metatheorem's exact scope

Here *first-order* has the meaning used in Henkin's construction: "there is" and "every" range over the individual objects supplied by the model. Our variables do not directly range over every possible collection of those objects or over all possible interpretations. When the objects themselves include sets, quantifiers may range over those sets as objects, with the range still supplied by the model.

Let T be a consistent classical first-order theory with computably enumerable certificates. Assume an effective sentence compiler $(e, w) \mapsto \mathrm{Out}(e, w)$, correct in the intended finite-record interpretation, and assume that every actual finite halting computation yields effectively a T-proof of its output sentence. The recursion theorem then produces E that searches for certificates of $\neg\,\mathrm{Out}(E, w)$ and reacts by producing w.

Fix the compiler and the certificate enumeration before constructing E. The task supplied to Kleene's construction receives a program text e, enumerates certified conclusions, and looks for one in the specified form $\neg\,\mathrm{Out}(e, w)$. The word w is recoverable from that syntactic form by the fixed coding convention. If a matching conclusion appears, the task writes that word and halts. Kleene supplies a complete program that performs this task with its own text in place of e; there is no extra axiom asserting that the program knows its text.

If any such certificate existed, the first one encountered would produce an actual finite run. Finite-trace verification would supply the opposite theorem, contradicting consistency. Thus $T \nvdash \neg\,\mathrm{Out}(E, w)$ for each standard w. By the deduction theorem, $T + \mathrm{Out}(E, w)$ is consistent. Consistency is our assumption about T; the proofs required within T concern the particular finite executions.

The hypothetical excluding certificate has a particular finite place in the enumeration. Follow the search as far as that place, including the reconstruction of the program's description, and then follow its response. That is the ordinary finite execution the finite-trace hypothesis would certify. No proof that the search always succeeds is required. Nor must T prove that the division programs always

terminate. The contradiction uses only this one response to this one hypothetical certificate.

To see the consistency conclusion directly in our calculus, suppose $T + \mathrm{Out}(E, w)$ proved $\bot$. Discharge every use of the extra assumption $\mathrm{Out}(E, w)$ by the negation rule. The result would be a T-proof of $\neg\,\mathrm{Out}(E, w)$, which the preceding argument excludes. This proves the conclusion for each prescribed ordinary finite word; it does not assert the simultaneous consistency of incompatible outputs.

The program J of Chapter 5 (main book) yields a true unprovable nonhalting assertion. The positive halting sentence may remain provable in a consistent theory. The broader result of essential incompleteness follows from AS's interpretation strength: every consistent computably enumerable extension of AS leaves some sentence undecided.

Chapter 3

Further instruction and proof examples

3.1 An endless printer, or a very patient search?

We might try to bargain with the halting problem. Forget every possible kind of program, with its own peculiar reasons for stopping or carrying on. We only want an inventory of decimal printers. Could an inspector tell which sheets keep supplying digits? It sounds like a considerable reduction in duties. A row of zeros will show why the problem has hardly become smaller at all.

Take any program whose stopping behavior interests us. Give a new worker the job of simulating it on separate working pages. Before each simulated instruction, the new worker checks whether the simulated program has reached its stop instruction. If so, the new worker stops too. Otherwise the new worker adds a zero to a decimal row beginning with 0., performs the next simulated instruction on those working pages, and repeats.

Keep the simulated working pages separate from the decimal output. The simulated program might write words, erase its own working rows, or print digits unrelated to our question. Those changes remain in its recorded paper. The public row belongs to the new worker, whose separate instruction is to add zeros. One simulated instruction always takes finitely many of the new worker's steps: the recorded configuration at that stage is finite.

For the three-line example, the new worker writes two zeros: one before simulating the write of 7, and one before simulating the move. The next recorded instruction is `Halt`, so the public output ends. For an endless backward jump, a further zero appears before every simulated jump. Its public output continues

$$0.000000\ldots.$$

In general, the new worker keeps producing digits exactly when the simulated program never stops. If an inspector could always

recognize endless decimal printers, it could inspect this new sheet and thereby answer the original halting question. We have just proved that no method can always do that. The desired printer inspector cannot exist.

Nothing in the row of zeros announces what it is measuring. Yet each new zero records another simulated step, and an endless row records a calculation that never stops. An inspector who could settle the question about digits would settle the hidden halting question too. We can hide it again, this time in the difference between rational and irrational numbers.

3.2 A rational number, unless a halt is found

Let the worker go on writing zeros while simulating the supplied program. So far the decimal is behaving impeccably: it looks like zero. At a simulated halt, change its future. Switch to the growing-gap instructions from Chapter 2 (main book): append one zero and a one, then two zeros and a one, then three zeros and a one, continuing to increase the gap. The already-written zeros stay where they are. Their infinite continuation has changed.

If the simulated program never stops, the result is zero, a rational number. If it has two working steps before halting, the output begins with the two waiting zeros and then the growing-gap blocks:

$$0.\underbrace{00}_{\text{waiting}}\underbrace{01\ 001\ 0001\ 00001\ \cdots}_{\text{growing gaps}}.$$

The spaces in this display separate blocks for our eyes. The worker writes one uninterrupted row of digits.

Why is the second outcome irrational? Long division taught us that every rational decimal eventually repeats a fixed block. A repeating block containing a 1 would put a bound on the lengths of all later gaps between ones. Our gaps exceed every fixed bound. A repeating block containing no 1 would leave no later ones at all, while our construction supplies infinitely many. Neither possibility works. The finite waiting zeros at the beginning do not affect either reason.

An inspector that always classified endless printers as rational or irrational could inspect this new sheet. A rational answer would mean that the simulated program never stops. An irrational answer would mean that it stops. Once again a supposed method for the new question would solve the halting problem, so no such universal method exists.

The apparently idle zeros were essential. Had the worker waited silently for a halt, an endless simulation would have left an empty row. An inspector required to judge only endless printers could quite reasonably refuse that sheet. The zeros keep digits arriving whether or not the simulation ever stops. There is no need to know which case we are in before presenting a genuine endless printer for inspection.

3.3 Taking apart a conjunction

Put $(a \in b) \wedge (b \in c)$ on the page. We have joined two facts, and would like them in the opposite order. Nothing about the bags needs to change. We only need to take the written statement apart and put its pieces back differently. The two taking-apart cards do the first job.

For instance, we can reverse the order of the two facts. First take out the right-hand fact:

$$\frac{(a \in b) \wedge (b \in c)}{b \in c} \quad \text{take right.}$$

Take out the left-hand fact as well:

$$\frac{(a \in b) \wedge (b \in c)}{a \in b} \quad \text{take left.}$$

Now join them in their new order:

$$\frac{b \in c \qquad a \in b}{(b \in c) \wedge (a \in b)} \quad \text{keep both.}$$

The last tree begins with the conclusions of the first two. The checker can follow the whole rearrangement by comparing pieces of writing: take the right part, take the left part, and match them with the new

conjunction. The bags could remain sealed throughout. This little proof is already an example of how checking marks can preserve a claim about things the checker never gets to examine.

3.4 The closing label

Try something temporarily and see what follows: a perfectly ordinary way to think, and a dangerous way to lose track of what we know. In Chapter 5 (main book) we began with $e \in b$ and $\neg(e \in e)$, tried $[b = e]_4$, and reached a contradiction. The final negation closed exactly that trial. The label tells a checker which supposition has come to an end.

Here is the check on the label 4 in that branch:

Line being checked	Assumptions kept open
$e \in e$ inside the branch	$e \in b$, $\neg(e \in e)$, and $[b = e]_4$
$\neg(b = e)$ after closing it	$e \in b$ and $\neg(e \in e)$

Closing label 4 removes that assumption from the list for the final line. It does not erase the two facts we began with. If the closing instruction named label 5 instead, label 4 would still be open. The checker would refuse to certify a conclusion presented as independent of it. Likewise, the interior line $e \in e$ cannot be carried out of the closed branch as an established fact.

3.5 An unknown witness and a captured name

From $a \in b$, the example rule gives $\exists x(x \in b)$. The assertion may also be available without any particular member having been named.

Now run the thought in the other direction. We know only that b has a member. We have not been told which object it is. For the duration of an argument we may call it u. Our temporary information is $u \in b$. From that information, we can give an example for the longer claim

$$\exists x \big((x \in b) \wedge (x = x)\big).$$

It says that b has a member that is equal to itself. Here is the little branch doing the work:

$$\dfrac{\dfrac{[u \in b] \qquad u = u}{(u \in b) \wedge (u = u)}\ \text{keep both}}{\exists x\,\big((x \in b) \wedge (x = x)\big)}$$

give example u.

The conclusion does not mention the temporary name u. It holds whichever member of b the name stood for. We may therefore close the branch and obtain that conclusion from the original assertion that b has a member.

Give the witness a fresh name. Reusing a, when a already names something we know about, could lend the unknown witness somebody else's properties. Suppose we know that the bag has a member and that a is outside the bag. Calling its unknown member a would let us announce a contradiction created entirely by our choice of letter. The bag has done nothing to deserve it.

The checker consequently scans three places: the original existence statement, the proposed conclusion, and the assumptions that will remain open when the witness branch closes. The temporary name must not occur in any of them. Its use belongs only to the branch that investigates the unknown witness. This is a condition about visible writing, so the worker can enforce it without knowing which member of the bag exists.

A name can also occur in a different grammatical role. In $\exists x(x \in b)$, the existential sign controls the occurrences of x inside its parentheses. We call those occurrences *bound*. The occurrence of b is *free*: the assertion still concerns whichever object we have called b. Replacing the bound x throughout by a fresh letter changes no claim, just as renaming the blank in an application form changes none of the information requested.

Here is how a careless replacement could change the claim. Suppose $b = x$: these two free names refer to the same bag. We also know $\exists x(x \in b)$, which says that b has a member. Replacing b by x without further care would give

$$\exists x(x \in x).$$

That says some object belongs to itself. It is a quite different assertion. The newly inserted x, intended as a name for our bag, has fallen under the command "there exists an x."

First rename the old bound letter to an unused letter z. Our statement becomes $\exists z(z \in b)$, still saying that b has a member. Now replace b by the equal object named x:

$$\exists z(z \in x).$$

The free x still names the bag, while the bound z ranges over possible members. Neither has taken over the other's job. Checking the written names prevents the accident; the safeguard is called avoiding *capture*. A problem about what a sentence means has left a trace that our worker can check on paper.

3.6 Representing numbers by other objects

The set language has been speaking about arithmetic. This is rather like discovering that the bags can count, until we look at what the translation actually requires. Only sets that pass a specified test represent numbers. The coding interpretation used in Chapter 5 (main book) supplies that test, together with translations of claims about numbers and computation records.

Let us try one small translation before using that bridge. In number language we say, "Every number has a successor." In the set language, we first need a test for which objects represent numbers. Write $\mathrm{Num}(x)$ for that test and $\mathrm{Succ}(x, y)$ for the test saying that y represents the successor of the number represented by x. The translation has this shape:

$$\forall x\big(\mathrm{Num}(x) \longrightarrow \exists y(\mathrm{Num}(y) \wedge \mathrm{Succ}(x, y))\big).$$

Start with any object x in the world. If it fails the number test, this sentence asks nothing more of it: a bag need not be a number code. If it passes, we must find another passing object y and check the successor relation. For a code representing 2, that means finding a code representing 3. Two different passing objects may represent the same number, just as different containers can represent the same pair.

Try a small example with ordinary cards before returning to sets. Allow cards of the form (n, red) and (n, blue), where n is an ordinary nonnegative integer. Our number test accepts exactly these cards. It rejects a card saying only "Monday." The cards $(2, \text{red})$ and $(2, \text{blue})$ are different pieces of data, but our equality test ignores their colours and compares their numbers: they represent the same number. Write EqNum for that test. The successor test also ignores colour. It accepts $(2, \text{red})$, $(3, \text{blue})$, and still accepts if either colour changes; it rejects $(2, \text{blue})$, $(4, \text{red})$. Thus translating "two equals two" does not require red to equal blue. It requires EqNum to accept the chosen two codes. This card game illustrates represented equality; it is not the set theory's actual coding scheme, which the interpretation theorem supplies.

In the set-language translation above, Num, EqNum, and Succ abbreviate fixed formulas using membership, equality, and the logical signs we have learned. The borrowed interpretation theorem supplies those formulas and the proofs of their arithmetic laws. Ordinary numerals name numbers in that domain, but a model's domain can also contain represented numbers beyond every ordinary numeral. A quantifier over represented numbers covers those too.

Chapter 4

Further rooms

An endless game can always have a winner without either player having a winning strategy. A countable mathematical world can insist, quite correctly by its own lights, that its reals cannot be counted. These are the sorts of things our drawers and notebooks have left us with. We can follow them a little farther here.

The outside list leads us back to the game; then we open the forcing names and examine how their choices become truths about a world. These excursions can be read independently after Chapters 3 (main book) and 6 (main book), and the coda.

4.1 A list visible only from outside

Henkin's construction puts the objects of a countable model within reach of a list. What if the theory we started with insists that its real numbers cannot be listed? Thoralf Skolem drew attention to this awkward meeting of inside and outside. Suppose a consistent, countable first-order theory describes sets, includes real numbers among its objects, and proves that its reals cannot be listed by its natural numbers. It has a countable model. We can list the model's objects from outside, including those it calls real numbers. Inside, the theory's statement still holds: no list enumerates all the reals.



Thoralf Skolem (1887–1963)

Number the objects from outside and circle those the model calls reals. We have made our list. Now try to hand it over. Nothing in the numbering has supplied an object inside the model that performs this enumeration. The model's assertion concerns its own lists, among its own objects. Holding our outside list up to the window does not put it inside.

This example uses additional set-theoretic axioms describing an entire set of reals, beyond the empty-set and adjunction axioms we used to encode finite calculations.

4.2 How much of a drawer can a strategy control?

The players still take turns appending any nonempty finite block of digits. The first wants the target event to occur; the second wants it to fail. Chapter 3 (main book) gave the second player a strategy when the target is *meager*: a numbered list of pieces that can each be escaped from every drawer. If failure is meager inside one drawer, the first player can start there and escape the failures instead. We called the target *comeager* in that drawer.

Perhaps an ingenious player could win by some entirely different means. To find out, we will take a strategy apart, saving some of its replies in separate drawers. A completed writing will then let us reconstruct a history of moves. There is a reason to be selective: the same row of digits might have arrived as two long turns or six short ones. Saving every reply would leave those histories tangled. The drawers we keep will tell us how the play actually went.

Suppose someone hands us a strategy and claims it always wins. We will examine its possible replies. For this argument the drawers collect written digit strings. A string beginning 2999 and a string beginning 3000 are different possible writings even when their completed values coincide. Both are allowed. A drawer fixes the beginning of the writing, which lets us recover moves as well as digits.

Freeze the play just after the strategy makes a move. The page says 25, and we have saved the history that brought it there. From this one moment the game can continue in many ways. The table tries some possible opposing moves and replies. These local replies

do not claim to win a particular event; they let us rehearse the selection. When an actual strategy is presented, we use the replies it prescribes.

opponent adds	reply	resulting drawer	selection
30	4	[0.25304]	keep
3	0	[0.2530]	reject: contains the first
304	2	[0.253042]	reject: inside the first
37	8	[0.25378]	keep: separate
8	1	[0.2581]	keep: separate

Each row rewinds to the saved 25 and tries another continuation. Reading down the table therefore explores alternatives; it does not play successive turns. Each alternative adds two nonempty blocks, as the rules require.

To continue the table, list every possible opposing block and the strategy's reply to it. We can put all finite blocks in a numbered list, as we did with instruction sheets. For each row, keep its drawer if it has no writing in common with a drawer kept earlier; otherwise reject it. Save both moves beside every drawer we keep. In the sample, a writing beginning 25304 directs us to the saved moves 30, then 4. A writing beginning 25378 directs us to 37, then 8. Since those drawers are separate, the two directions cannot conflict.

Let us check the rejections with a pencil on the prefixes. Every writing beginning 253042 also begins 25304; rejecting the third row leaves all of it covered by the first. In the second row the containment goes the other way: 2530 is shorter, so its drawer contains the retained 25304 drawer. We have kept some room inside it, but not all of it. What about the smaller drawer [0.25309], which that retained row does not reach? It has its own opportunity. The opponent can write 309 after 25. Whatever the strategy then replies, that new row ends inside [0.25309]. The complete list includes this row too. We must continue through the list before judging what room the retained collection reaches.

Here is the general check behind that example. Choose any smaller drawer J inside [0.25]. The opponent can extend 25 into

J; append an extra digit if needed to make the move nonempty. The strategy's reply then reaches a listed drawer L contained in J. If L is kept, we have found a whole drawer inside J in the retained collection. If it is rejected, it overlaps a drawer K kept earlier. Overlapping prefixes must extend one another. There are only two cases:

- If K is inside L, use K. It lies in J and is itself retained.
- If L is inside K, use L. It lies in J, and every writing in it belongs to the retained drawer K.

Either way a whole smaller drawer in J enters the retained collection. We have proved that rejection leaves no smaller drawer without a way in. This is the precise sense in which the retained collection is *dense* in our starting drawer: wherever a finite prefix takes us, some further finite extension enters that collection.

We have left a way into the retained collection from every smaller drawer. That does not mean every completed writing enters it. A writing could miss every retained drawer: call that the first failure event. Wherever we are inside [0.25], however, a finite extension can make this failure impossible. It is nowhere dense there, in the sense used in Chapter 3 (main book). The difference between "a way in from everywhere" and "everyone is already in" will keep doing work.

Now perform a second generation of the same work. Start inside each retained drawer, with its saved history. Inside [0.25304] that history includes the opponent's 30 and the strategy's 4. Possible next pairs of moves might begin as follows:

opponent adds	reply	resulting drawer	selection
10	6	[0.25304106]	keep
1	0	[0.2530410]	reject: contains the first
106	2	[0.253041062]	reject: inside the first
7	0	[0.2530470]	keep: separate

Again these are sample replies; we continue through all opposing moves. We do separate work under the retained drawer [0.25378],

starting with its own saved moves 37, then 8. There, for example, an opposing 2 and a reply 5 could give [0.2537825]. Replies are always consulted at the history to which they belong.

Follow one writing through the two generations:

$$\underbrace{25}_{\text{starting history}} \quad \underbrace{30}_{\text{opponent}} \quad \underbrace{4}_{\text{strategy}} \quad \underbrace{10}_{\text{opponent}} \quad \underbrace{6}_{\text{strategy}} \quad \cdots .$$

Its first selected drawer is [0.25304]; its next is [0.25304106]. The second record retains the first pair of moves and adds the second pair. It does not choose a different way of arriving at 25304. Thus these saved histories fit together into one legal play. A writing beginning 2530470 would use the same first pair and then 7, 0 instead.

Could the second generation leave an entire smaller drawer untouched? Begin anywhere inside the original [0.25]. The first-generation check supplies a further drawer wholly inside one retained first drawer. Apply the same check there to its second-generation selection. A further extension enters a retained second drawer. So the event of missing all second-generation drawers is also nowhere dense in the original drawer. The reasoning repeats for each generation: first get inside a saved drawer from the previous generation, then use the selection made there.

We now have a numbered list of failure events, one for each generation. Their union is meager. Consider a completed writing outside that union. It enters exactly one retained drawer at every generation: at least one because it avoids that generation's failure, and at most one because retained drawers are separate. Each chosen drawer belongs beneath the previous one. Its saved history extends the previous saved history, just as 30, 4, 10, 6 extended 30, 4 in our example. Every pair adds digits, so these histories together produce the whole completed writing, with no unfilled tail. They describe a legal endless play following the strategy.

Only now do we use the boast that the strategy always wins. We have reconstructed a genuine play that follows it, so that play must meet its winning condition. Every writing outside our meager list of failures arises in this way. Thus success fills the starting drawer

apart from a meager event. Taking the strategy's replies apart has revealed how much of a drawer its promise commits it to winning.

For the first player, the opening move chooses that drawer. A winning strategy makes the target event comeager there. For the second player, apply the selection first to every possible opening by the opponent and the strategy's reply. Given any drawer, the opponent can open inside it; the reply stays inside it. The same retention check therefore gives a dense first collection in the whole digit space. A winning strategy makes failure of the target comeager everywhere; the target must be meager.

The two descriptions now fit exactly. The second player can guarantee failure just when the target event is meager. The first can guarantee success just when the target is comeager inside some drawer. Behind those compact words lie the finite choices we have been saving and reconstructing. They tell us what a guaranteed victory would require.

4.3 A game with no guaranteed winner

We can now invent an event that gives neither player the advantage we just identified. Its construction uses the *axiom of choice*: from any family of nonempty collections, one may select a member of each. The selection is allowed to exist even if no finite sheet could tell us how to make it. A choice without directions is exactly what makes the coming game troublesome.

Group digit strings together when they differ in only finitely many places. For example, $314159\ldots$ and $004159\ldots$ belong to the same group if all their later digits agree. Strings that differ at infinitely many places belong to different groups. Every string belongs to exactly one group.

Choose one string from each group and collect the choices in S. The first player now wants the completed play to land in S. That is a perfectly definite event, though the axiom has provided no machine for recognizing it. We know how many representatives each group contributes—exactly one—without having a set of directions for pointing to them.

Consider a simple way to change a string: rotate its first digit one place around the circle

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0.$$

Or rotate the third digit by two places, leaving all the others alone. More generally, prescribe a finite list of positions and a rotation at each. Each prescription can be undone, and it changes only finitely many digits. There are countably many prescriptions because each has a finite written description.

Apply one prescription to every member of S. The result is a changed copy of S. Include the prescription that changes nothing. These copies cover all possible strings: any string differs in finitely many places from its group's selected member, and the right rotations turn that member into it.

These changed copies cannot share a member. If two did, undoing the rotations would uncover two members of S differing only finitely. They belong to the same group, which was allowed just one choice, so they must be the same selected member. But different rotations of those same digits give different results. The supposed common member is impossible. We record each affected position once and use one of the nine nonzero rotations, so two prescriptions cannot be different spellings of the same change.

A finite rotation also preserves our possibilities for escape. It turns a drawer into another drawer by changing the specified digits in its prefix; rotations at later positions leave that drawer as a whole unchanged. If an event can be avoided inside every drawer, its rotated copy can be avoided too: undo the rotation, make the escape, and rotate back. The same holds for a numbered list of such events.

Now suppose S were meager. All its rotated copies would be meager too. There are countably many copies, and each has a countable list of escapable pieces. Put the lists together, for example by attending to the first piece on the first list, then the first two pieces on the first two lists, then the first three on the first three, and so on. This covers every copy by one countable list of escapable pieces. But the copies cover every string. Baire says that such a list cannot cover even one drawer. Thus S is not meager.

Could S be comeager inside some drawer? Suppose that drawer fixes the first k digits. Rotate digit $k+1$ by one place. The rotation stays inside the drawer and takes S to a disjoint copy. If S fills the drawer except for a meager failure event, so does its rotated copy. Combine the two failure lists. Baire gives a string in the drawer avoiding both failures, so it would belong to both copies. The copies are disjoint. This second contradiction shows that S is comeager in no drawer.

Neither player has a winning strategy. The second would need S to be meager; the first would need it to be comeager in some drawer. We have denied them both. Every completed play nevertheless has a winner: its string is in S, or it is not. The game gives a definite answer after a whole play without giving either player a way to guarantee that answer in advance, even with unrestricted mathematical ingenuity.

The game can also be made to concern the real value of the decimal. Remove from S the strings eventually consisting entirely of zeros or entirely of nines. There are only countably many of these, since a finite beginning and a choice of tail specify one. Removing countably many strings cannot turn a nonmeager event into a meager one: adding them back would then be a countable union of meager events. Nor can removal make an event comeager in a drawer where it was not already comeager. The two obstructions remain.

Each remaining selected string names a real with only one decimal expansion. Declare the first player's target to be those real values. Both expansions remain legal in the game; a value with two expansions is simply outside this particular target. The winning-strategy obstructions therefore hold for an event about real numbers as well.

The obstruction survives the move from written strings to real values. We allowed strategies far beyond anything a pencil worker could compute, and still neither player could secure a win. This time the limitation belongs to the game itself.

4.4 A workshop for forcing names

Take out the two-row card from the coda. We are going to open it, put one of its names inside another, and watch which membership claims survive each choice of digit. The conditions are still finite zero–one strings. Eventually we use a choice generic over the countable transitive old world described there. First we can see quite a lot by following the rows with a pencil.

4.5 A name is a set of conditional instructions

Before making the card precise, let us give its numbers a place among sets. We shall use the empty set as zero, and the set containing just zero as one:

$$0 = \varnothing, \qquad 1 = \{\varnothing\} = \{0\}.$$

Zero has no members; one has one member, namely zero. These are choices of set objects to represent the first two integers. A written digit on our paper still does its familiar job of specifying a choice. The set named by that digit does a different job: it can be included as a member of another set. The column headings on our cards will keep those jobs separate.

Our opening card suggests the general pattern. A name is a collection of pairs. In each pair, the first entry is another name and the second is a condition. The pair means

> Include the object named by the first entry if the condition in the second entry is one of our chosen prefixes.

Names are built from earlier names, so we can understand a large one by first understanding the smaller names it mentions.

Start with a name that has no pairs. It has no instructions to include anything, so its value is the empty set. Next pair that name with the empty condition. The empty condition is always accepted. This new name therefore always includes the empty set: its value is $\{\varnothing\}$. The distinction between the empty set and a set having the empty set as its one member is exactly the distinction from our adjunctive examples.

An object a that we already have receives a name whose value is fixed, written $\check{a}$. For each member of a, include its corresponding name with the empty condition. All these membership instructions take effect, giving back exactly a. The small check over the letter means "the standard name for this old object."

Return to our opening card and write its instructions as pairs. The name is

$$\tau = \{(\check{0}, 0), (\check{1}, 1)\}.$$

The second entries here are one-digit conditions. Call the eventual sequence g. If g begins with zero, the first condition is accepted and the second is not. The value of τ is then $\{0\}$. If g begins with one, its value is $\{1\}$. We can check either case by marking the applicable row on our opening card. The whole card sits on the desk while the first digit remains unwritten. Its directions are complete; its value is still waiting for the choice.

Call the collection of finite prefixes of g by the name G. If τ^G means the value of the name after those prefixes have been chosen, the general instruction is written

$$\tau^G = \{\sigma^G : (\sigma, p) \in \tau \text{ and } p \in G\}.$$

To read it, look at each pair (σ, p) in the name. If p is accepted, include the value of σ; otherwise ignore that pair. Repeated instructions to include the same object make only one member. The formula says exactly what the little card did.

For example, suppose our chosen sequence begins 010. Substitute the two rows into the test. The row $(\check{0}, 0)$ passes because 0 is a chosen prefix. Its first entry has value $(\check{0})^G = 0$, so include zero. The row $(\check{1}, 1)$ fails because 1 is not a prefix. There is just one accepted value, and collecting it into a set gives $\tau^G = \{0\}$. The braces perform the last step: they collect the accepted objects, rather than changing those objects into something else.

There is also a name for the new sequence viewed as a set of positions: include the old integer n under every condition that writes a one in position n. Once G is chosen, its value is precisely the set of positions at which g has a one. The name specifies what to include at each position without first settling the whole sequence.

A name can contain infinitely many instructions. Its component names must be evaluated before the whole, just as our empty name was evaluated before the name of its one-member set. The usual set axioms organize these dependencies without an endless descent through smaller and smaller components. They give a mathematical construction, not a promise that a worker will finish the inspection. The tidy label on a name is no guarantee of tidy contents. It may not even supply a program for listing all the conditional instructions. We met this separation with Henkin: a name lets us refer to an object without first calculating everything about it.

4.6 What a condition can already settle

Return to the name τ with its two rows. The condition 0 already settles that zero belongs to its value. Every further compatible choice leaves the first digit zero. The condition 1 settles the opposite. The empty condition settles neither, although it settles that the resulting set is either $\{0\}$ or $\{1\}$.

Forcing develops a relation saying which sentences a condition settles. The notation

$$p \Vdash A$$

is read "p forces A." In our example, the left side is the finite information already written; A is a claim about the values of names. Stronger compatible conditions retain what has already been forced.

Our card lets us work out these assertions before constructing any new world. For example,

$$0 \Vdash 0 \in \tau, \qquad 1 \Vdash \neg(0 \in \tau).$$

We suppress the check marks on the fixed names for zero here. The first condition guarantees that the zero row is included. The second leaves no compatible way of including that row. The empty condition cannot guarantee either assertion, because it still allows both first digits.

Before choosing the first digit, we can already guarantee that τ has a member. A zero gives us zero as a witness; a one gives us one.

Either way, a further commitment will identify somebody in the set. The two rows have supplied a proof by cases. The existence claim is ready before we have picked its example.

We can even prepare one conditional name for the witness. Use the set numerals $0 = \varnothing$ and $1 = \{0\}$ just introduced. Take the one-row name

$$\rho = \{(\check{0}, 1)\}.$$

If the first digit is zero, this row is not accepted and ρ has value $\varnothing = 0$. If it is one, the row is accepted and ρ has value $\{0\} = 1$. Thus ρ names a member of τ in either case. Its value changes with the choice, while its job as a witness remains fixed.

Here are both calculations side by side. The one row of ρ says "include the object named by $\check{0}$ when the required prefix is 1."

first digit	value of τ	accepted rows of ρ	value of ρ
0	$\{0\}$	none	$\varnothing = 0$
1	$\{1\}$	its one row	$\{0\} = 1$

Now put that whole name on a new card. Write e for the empty condition and set

$$\eta = \{(\rho, e)\}.$$

There is only one row to inspect, and its condition is accepted on either branch. But the first entry is ρ, a name whose value we still have to calculate. Follow the zero branch first. The row of η is accepted; the row inside ρ is rejected; so ρ contributes the empty set. Collect that one contributed object into a set:

$$\rho^G = \varnothing = 0, \qquad \eta^G = \{\varnothing\} = \{0\}.$$

On the one branch, the outer row is again accepted. This time the row inside ρ is accepted too, so ρ contributes $\{0\} = 1$. Collecting that object gives

$$\rho^G = \{0\} = 1, \qquad \eta^G = \{\{0\}\} = \{1\}.$$

We accepted the same outer row both times and obtained different members. The row had told us to include the value of ρ, which itself depended on the choice. This gives us two jobs to keep separate:

test the row's condition, then evaluate the name on that row. Only after finding the included objects do we put braces around them. The cards η and τ are different, but following either card on a given branch produces the same set.

For more complicated names, forcing defines the same relation by working through their component names and the parts of a formula. Conjunction has the expected rule: a condition forces both statements when it forces each of them. Negation asks about all stronger conditions: it forces "not A" when no stronger condition can force A. An existence assertion requires that, however the condition is strengthened, a further strengthening can supply a name for a witness. Try the existence rule as a small challenge. We claim, before any digit is written, that τ has a member. Someone else first extends our blank condition however they like. If their extension begins with zero, show the fixed name $\check{0}$. If it begins with one, show $\check{1}$. If they have written nothing, extend by zero and show $\check{0}$. In every case we can make a further commitment, allowing the condition already reached to count as its own extension, and identify a member there. The order matters: *every* challenge must have *some* response. We did not have to choose the same fixed old member before hearing the challenge. The conditional name ρ packages those different responses into one name.

Compare this with the claim "zero belongs to τ." At the blank condition it is unsettled. Choosing zero can still make it true, and choosing one can make it false. After choosing one, however, no extension can restore the rejected zero row. Only then have we forced its negation. Leaving a question open and ruling out its positive answer are different actions.

Our cards have let us check membership, negation, and a witness with our own hands. The full definition follows membership and equality through arbitrary names, and a substantial theorem connects this forcing relation with truth. The source notes give places to follow those proofs. We will use the results to assemble the notebook; the small calculations here show what its entries are doing.

4.7 Truth before a choice has been made

For a moment, resist answering "zero belongs to the set" with yes or no. Draw the part of the choice space that guarantees a yes: the drawer of sequences starting with zero. The opposite drawer starts with one. Between them they hold every possibility, and they never overlap. We can keep this picture while the first digit is still undecided.

Try a second statement: "the second digit is one." Its guaranteeing region consists of the two drawers beginning 01 and 11. For both statements together, keep only the common region, the drawer 01. For either statement, keep their combined region, the three drawers 00, 01, and 11. For neither, keep 10. We can do the logic by sorting four small drawers.

These regions are our first *Boolean truth values*. They tell us under which alternatives a claim is guaranteed. A region is not a numerical probability, and a sentence has not become partly true. After a suitable complete choice it will have an ordinary yes-or-no answer. Before the choice, the region tells us which possible answers belong where.

For infinite combinations, these values will describe guarantees for choices that meet the dense requests, not for every imaginable path. Such choices are called *generic*; they must meet the old world's dense requests, as in the coda. The difference appears as soon as we ask whether a one occurs somewhere after the second position. It could occur in position three, or four, or five, and so on. The drawers that already display such a one form an endless collection. Inside [0.00], for example, they include

$$[0.001], \quad [0.0001], \quad [0.00001], \quad \ldots.$$

Here the brackets name drawers of zero–one sequences. A sequence which remains zero forever belongs to none of these smaller drawers. Yet no finite string of zeros can guarantee that exception: a one can always be appended. More generally, every finite beginning can be extended to display a one after the second position.

Here is a drawer rule for doing the infinite combination in this presentation of forcing. First take the literal union of the witnessing drawers and call that region U. Then test a drawer as follows:

> However far someone narrows this drawer, can we narrow it further into a witnessing drawer in U?

If the answer is yes, U is dense inside the tested drawer. Collect all the drawers that pass this test. Their union is the *completed* value of U. The starting region must be built as a union of drawers: each of its sequences already has a finite beginning witnessing membership. Our later-one region has exactly that form.

Apply the rule to our later-one question. Test the drawer [0.00]. Someone narrows it to [0.00000]. We answer with [0.000001], wholly inside U. A longer string of zeros would only move our appended one farther along. If the challenger has already written a one after the second position, their drawer itself is in U. The same answer works from *any* starting drawer: extend to at least two positions if necessary, then append one. Even the whole space, the drawer with no digits specified, passes. The completed value is therefore the whole space. This has not made the all-zero path display a one. That path fails the dense request to write a one after position two. A choice meeting the request has an actual finite prefix displaying the one; the completed value records that guarantee for generic choices.

Change the question slightly: "The first digit is zero, and a one appears after the second position." Call its literal witnessing region V. It includes [0.001] and [0.0101], for example, but no drawer beginning with one. Inside [0.0], repeat the previous calculation: however someone extends the beginning, append a one after position two if none has yet appeared. The zero drawer passes. The one drawer fails, because no further writing can change its first digit. The whole space fails too: the challenger can narrow it to [0.1], where our request has become impossible. Every smaller drawer lies under one of those two first-digit drawers, so the completed value of V is exactly [0.0]. A generic choice beginning with zero supplies the later one; a choice beginning with one has already made the statement false.

Our two completed regions, the whole space and the zero drawer, are examples of a general system of operations on truth values. "Either" uses a *join*; "both" uses a *meet*; "not" has a matching operation. When those operations are supplied for every family

under consideration, the system is called a *complete Boolean algebra*. Inside an old model, "every family" ranges over that model's families. Skolem's lesson returns: an outside collection need not be one of the objects available inside.

We have calculated two infinite joins by the drawer rule. Arbitrary meets also require the completed operations; the finite example with four drawers is not a license to use literal intersections for every infinite family. Constructing and verifying the full system is part of the general forcing theory. Interpreting sentences in these values gives the *Boolean-valued* presentation of forcing. The finite four-drawer calculation survives, while the completed operations also handle existence questions whose witnesses may lie arbitrarily far ahead.

We can now discuss relationships between names before choosing their values. The drawers show where a claim is guaranteed; the completed operations let us combine such guarantees. To give the names values together, we must make a choice that meets all the old world's requests.

4.8 Henkin's notebook appears again

Give the old-world names finite labels and begin a fresh notebook in our first-order language. An accepted condition that forces a sentence earns that sentence a place in the notebook. Can the growing collection contradict itself? Take finitely many entries. Their conditions all begin the same g, so the longest extends every shorter one. It supports the selected sentences together, and the forcing rules prevent a contradiction. Any proof of contradiction would have used just such a finite selection. The whole notebook is consistent.

The forcing rules give dense collections of conditions that decide each sentence. Our generic choice meets those collections, so the notebook gives a yes-or-no answer to every sentence of the name language. Existence has named witnesses too. Our two-row card supplies a small example already on the desk. The empty condition guarantees that τ has a member, and the name ρ chooses zero or one as required. If a condition guarantees that some object has

a property, the forcing construction can supply a name for such an object. Different later choices may give that name different values, just as our two-row name can give either $\{0\}$ or $\{1\}$. The result assembling these possible witnesses is called the maximum principle; here it uses choice in the old world. Thus the notebook has the three features Henkin used. Its sentences do not contradict one another; every question receives a yes-or-no answer; and an assertion of existence has a named example.

For the branch we followed, the new notebook contains

$$\tau = \{\check{0}\}, \qquad \rho = \check{0}, \qquad \rho \in \tau.$$

The first digit already forces these sentences, so every later prefix supports them too. When we group equal names as Henkin did, τ joins the old name $\{\check{0}\}$, and ρ joins $\check{0}$. The membership sentence draws a link from the object named by ρ to the object named by τ: zero belongs to its singleton. We can check the link a second way by evaluating the accepted rows of the original card. Both readings give the same objects and the same membership fact.

Starting with one would instead put $\tau = \{\check{1}\}$ and $\rho = \check{1}$ in the notebook. The card was fixed throughout; the accepted conditions determined which equalities became true.

We group all names that the notebook says are equal, then use its membership sentences to define membership between the resulting objects. The truth lemma for forcing identifies this construction with the evaluated names: a sentence is true of their values exactly when some condition in G forces it. This is the bridge between the combinatorics of conditions and the truth of the notebook. The resulting world is written

$$M[G] = \{\tau^G : \tau \text{ is a forcing name in } M\}.$$

It includes every old object, because $\check{a}$ still evaluates to a, and it includes our new set of positions. Names organize the accompanying sets as well as the one new sequence.

We can cross this bridge by hand for our few names and sentences. The truth lemma establishes the crossing for every name and formula. The external G decides which sentences enter the

notebook; there is no promise that a mechanical proof search can list them. As before, the existence of the completed object has outgrown our ability to finish its pages.

We still owe the new world its set axioms. A preservation theorem says that this collection satisfies the usual axioms when the starting model does. For the empty set, our empty name already does the job. For a pair of named objects, put their two names on a card under the empty condition; both objects will be included. More demanding axioms need more demanding name constructions. These constructions do the work of keeping a whole world around the new sequence. Merely adding the sequence would not have paid that bill.

We can now check a small independence statement. Begin the construction with 0. Every dense request can still be met below that condition, so a completed notebook can say that the first digit of the new sequence is zero. Begin instead with 1, and the same construction gives a notebook saying it is one. The common requirements do not decide that digit. Each possible commitment can be supported by a complete model.

Our small independence proof ends with two settings satisfying the same requirements and giving opposite answers. It concerns the newly named sequence. To settle a question about independence in the old set-theoretic language, we need conditions whose combinatorics control that particular old-language statement. The famous applications of forcing do this further work. Our first digit shows the shape of the argument: each commitment leaves room for a whole world to follow it.

Sources and Further Reading

The workers, games, and mathematical worlds in these chapters have longer lives in the sources below. Some entries lead back to the original papers; others open up arguments that this book has kept close to the pencil and the decimal. The historical accounts also restore something an instruction sheet leaves out: the people who thought to write it.

The worker, self-reference, and formal proof

Sarah McLennan and Mary Gainer, "When the Computer Wore a Skirt: Langley's Computers, 1935–1970", NASA historical account. The first Langley computing pool belonged to NACA, NASA's predecessor. These professional computers performed skilled work, including checking calculations and developing methods. Our rule-following worker isolates one part of calculation; the expanding graph-paper desk is a mathematical model, not a reconstruction of their workplace.

Princeton's "An Early History of Computing at Princeton" reproduces Max Newman's contemporary description of Turing's independent work and discusses his subsequent Princeton years. Church's students enter this book in the order needed by its mathematical argument, not in the order of their doctorates. Turing did not develop his 1936 account as a member of a joint project with Kleene and Henkin.

Alan M. Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem," *Proceedings of the London Mathematical Society* 42 (1936–37), 230–265. Turing opens with the idea of calculating a real's decimal by finite means; his formal computing machines in Section 2 produce binary digits. He separates the lasting output from erasable working symbols and distinguishes a finite machine description from its endless output. Chapter 1 (main

book) relates a worker using graph paper and recorded places to a tape machine. Original paper, digitized copy.

Jeffry L. Hirst, "Representations of Reals in Reverse Mathematics," *Bulletin of the Polish Academy of Sciences. Mathematics* 55 (2007), 303–316, doi:10.4064/ba55-4-2. Hirst compares rapidly converging rational approximations, decimal expansions and Dedekind cuts. Sections 2–3 distinguish conversion for an individual real from a uniform conversion procedure. The computable reals form the same class under the digit and effective-approximation descriptions, while a general algorithm converting arbitrary effective approximations into a settled decimal expansion is unavailable. The distinction concerns the representations and the information supplied to a procedure. Publisher's full text.

David Hilbert, "The Foundations of Mathematics" (1927), translated in Jean van Heijenoort, ed., *From Frege to Gödel*. Hilbert treats formulas as concrete objects and derivations as operations governed by explicit rules. The proof checker follows this approach using natural-deduction proof trees. David Hilbert and Wilhelm Ackermann, *Grundzüge der theoretischen Logik* (1928), is a historical source for the first-order decision problem.

Stephen C. Kleene, *Introduction to Metamathematics* (1952), develops the classical computability and recursion-theorem framework. Kleene's Princeton oral-history interview gives a first-person account of Church's role. Chapter 4 (main book) proves the self-description construction used here with explicitly distinguished program builders and execution.

Zlatan Damnjanovic, "Mutual Interpretability of Robinson Arithmetic and Adjunctive Set Theory with Extensionality," *Bulletin of Symbolic Logic* 23 (2017), 381–404. The result covers adjunctive set theory with or without extensionality. doi:10.1017/bsl.2017.30. This establishes the strength of the two-axiom theory Empty and Adjunction used in Chapter 5 (main book). The chapter's uniform coding of computation records also uses the sequence machinery described in the next entry.

Albert Visser, "Pairs, Sets and Sequences in First-Order Theories," *Archive for Mathematical Logic* 47 (2008), 299–326, doi:10.1007/s00153-008-0087-1. The interpreted sequence machinery supplies uni-

form coding for finite records. Visser's "The Arithmetics of a Theory," *Notre Dame Journal of Formal Logic* 56 (2015), 81–119, doi:10.1215/00294527-2835029, develops compatible interpreted arithmetic and sequence structures. These papers give the full interpretation constructions; Chapter 2 of the *Technical Companion* identifies the coding properties needed for computation records.

Category, games, and worlds

René Baire, *Sur les fonctions de variables réelles* (1899), digitized dissertation. Chapter 3 (main book) develops category through events in a game that forms an infinite decimal. Drawers collect digit strings with a given finite beginning. Both decimal names of a shared numerical endpoint remain available; an event concerning a real number includes every address of that number. The successive-escape proof yields uncountability as a special case of Baire's category argument.

Baire's "Nouvelle démonstration d'un théorème sur les fonctions discontinues", *Bulletin de la Société Mathématique de France* 28 (1900), 173–179, revisits the work of his thesis on limits of continuous functions. It supplies the historical setting for the discussion of continuity and exceptional sets. Our decimal game presents a consequence of the method; it is not an account of how Baire discovered it.

R. Daniel Mauldin, ed., *The Scottish Book: Mathematics from The Scottish Café, with Selected Problems from The New Scottish Book*, second edition (2015), doi:10.1007/978-3-319-22897-6. Problem 43 and its commentary record Mazur's interval game and Banach's answer. In the decimal game of Chapter 3 (main book) and Chapter 4 of the *Technical Companion*, players choose finite digit blocks of variable length. The category characterization applies to that rule of play.

Leon Henkin, "The Completeness of the First-Order Functional Calculus," *Journal of Symbolic Logic* 14 (1949), 159–166, doi:10.2307/2267044. His retrospective "The Discovery of My Completeness Proofs" (1996), doi:10.2307/421107, explains the influence of Church's teaching and writings. Chapter 6 (main book)

gives the countable model-existence construction and derives compactness from it.

Asaf Karagila, *Forcing & Symmetric Extensions*, lecture notes, especially the opening sections on names and generic models. These develop the formal forcing relation, forcing theorem, and preservation results used in the coda. The coda and Chapter 4 of the *Technical Companion* assume a countable transitive ground model and construct a Henkin-style quotient of names. The ground-model assumption is additional to consistency of adjunctive set theory.

D. A. Martin, *Set Theory*, Math 223B lecture notes (2005), pp. 25–26, Lemma 3.7. The regular-open construction gives the completed Boolean operations. Chapter 4 of the *Technical Companion* expresses its join operation through the test of density inside a drawer and works two examples.

The final construction

Joel David Hamkins, “Every function can be computable!”, presents an alternative proof based on a suggestion of Vadim Kosoy: a program searches for a certificate excluding a finite pattern of its own behavior and is prepared to realize that pattern. Hamkins states that proof for a theory interpreting Peano arithmetic. Our formulation instead isolates the finite-verification and record-coding hypotheses it uses, with the adjunctive interpretation route described in Chapter 2 of the *Technical Companion*; that route is a separate dependency. This is the certificate reversal adapted in Chapter 7 (main book). Hamkins’s “The universal algorithm: a new simple proof of Woodin’s theorem” explains the stronger universal enumeration phenomenon. See also *The Modal Logic of Arithmetic Potentialism and the Universal Algorithm*, arXiv:1801.04599.

Chapter 7 (main book) adapts certificate reversal to one common quotient record, using the stated verification hypotheses and compactness. Its model realizes every standard prefix of a prescribed decimal; it need not extend a preselected model. The coda explains why ordinary forcing preserves existing computations, unlike the variation of nonstandard records used in the finale.